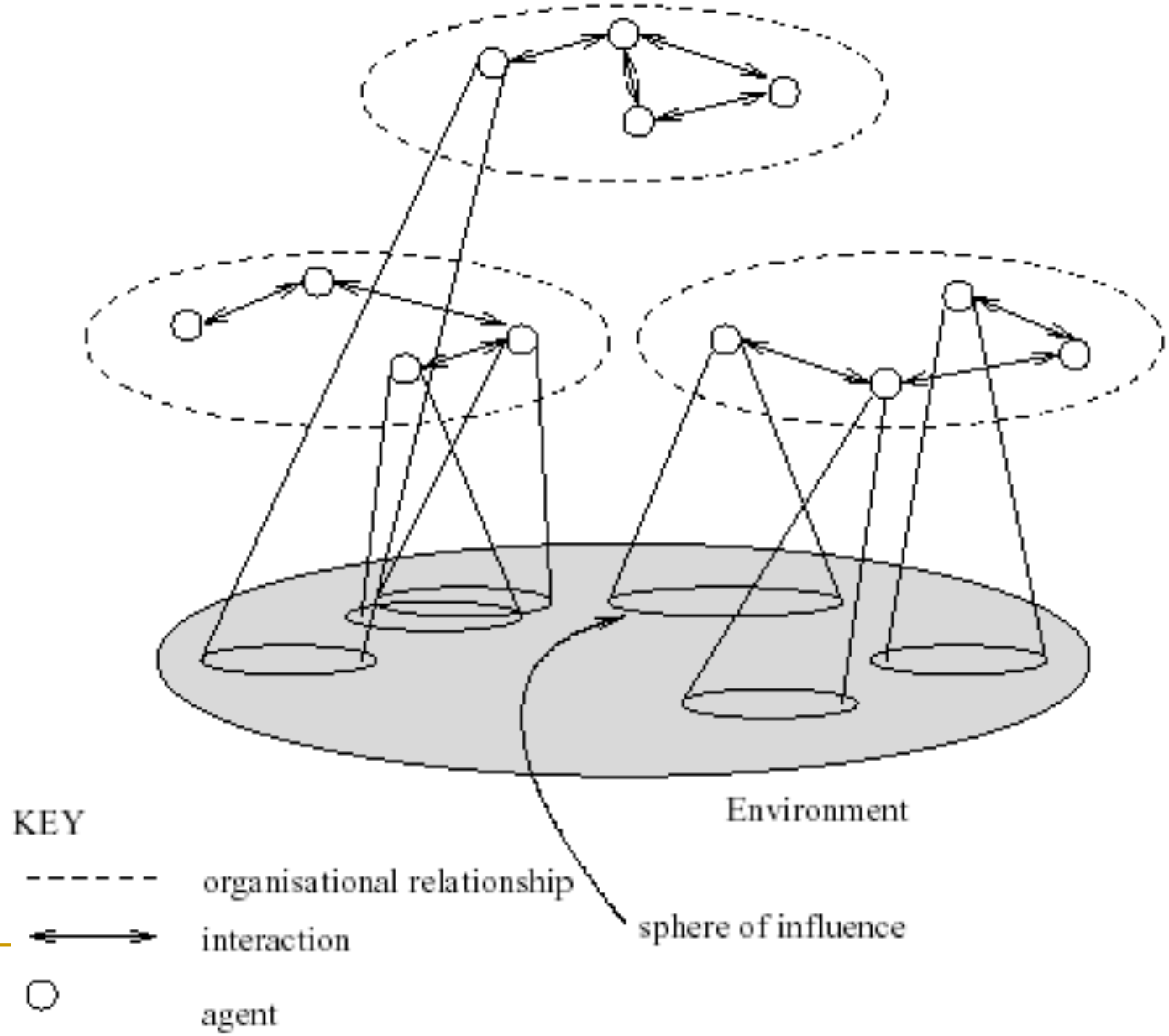
Ευφυείς Πράκτορες και Ρομποτικά Συστήματα

Γεώργιος Βούρος

MULTIAGENT INTERACTIONS



MultiAgent Systems

Thus a multiagent system contains a number of agents…

- …which interact through communication…
- …are able to act in an environment…
- …have different “spheres of influence” (which may coincide)…
- …will be linked by other (organizational) relationships

MultiAgent Systems

How agents collaborate to achieve a common goal?

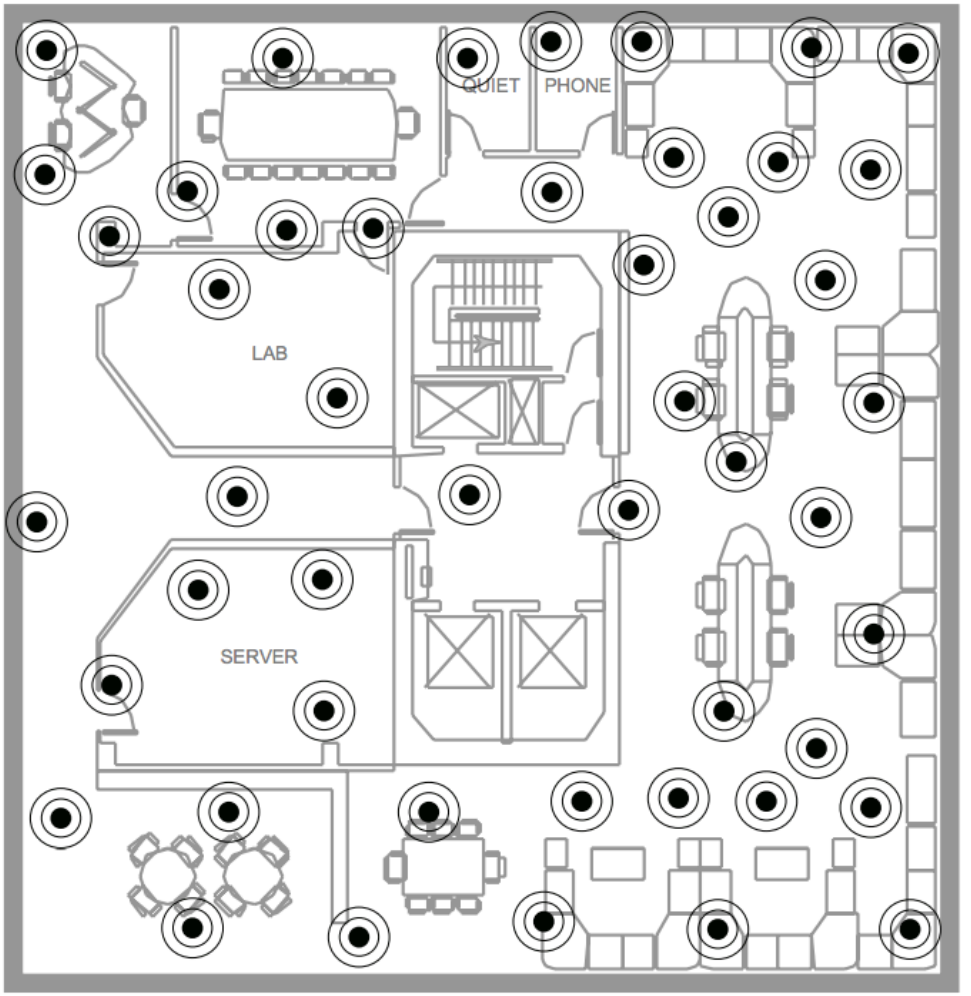
- No central controller
- Distributed resources

MultiAgent Systems

Example: Sensor Networks

- Sensor: local processing unit with limited processing power, local sensor capabilities, limited power supply and communication bandwidth

Goal: Provide a global service



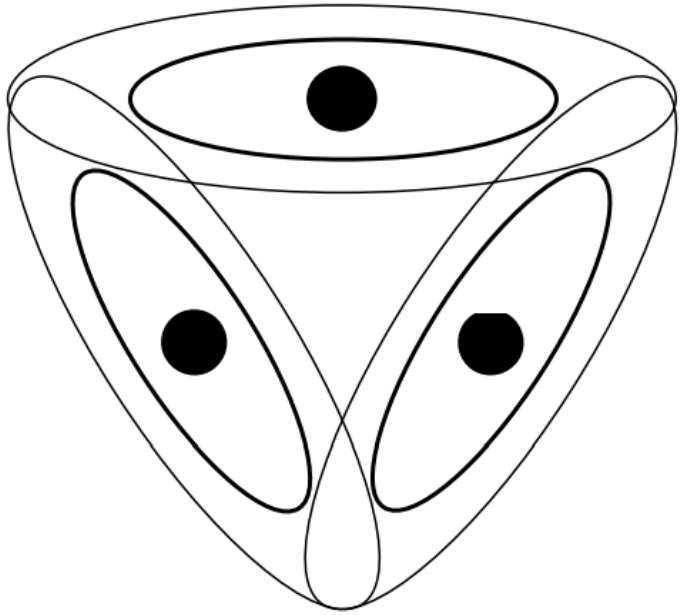
Distribution

- Distributed Constraint Satisfaction using **global** constraints
- Distributed Optimization (subject to global constraints)

Distributed Constraint Satisfaction

- Constraint Satisfaction Problem
  - Variables
  - Domains (per variable)
  - Constraints

Distributed Constraint Satisfaction



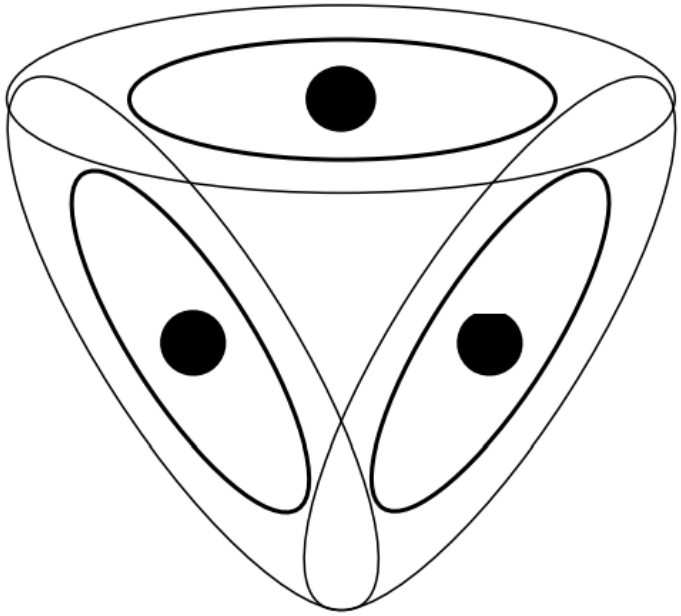
- **3 sensors example**

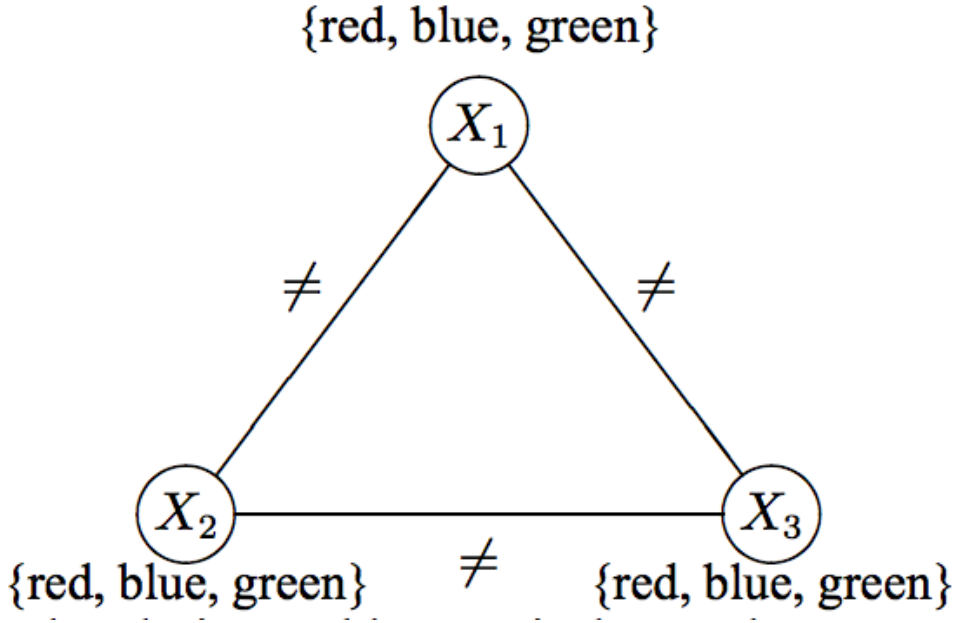
Suppose that each sensor can choose one of three possible radio frequencies.

All the frequencies work equally well so long as no two sensors with overlapping coverage areas use the same frequency.

Question: Which algorithms the sensors should employ to select their frequencies, assuming that this decision cannot be made centrally.

Distributed Constraint Satisfaction





Distributed Constraint Satisfaction

- Constraint Satisfaction Problem
  - Variables
  - Domains (per variable)
  - Constraints
- Solution: Legal instantiation of variables (i.e. no constraints are violated)

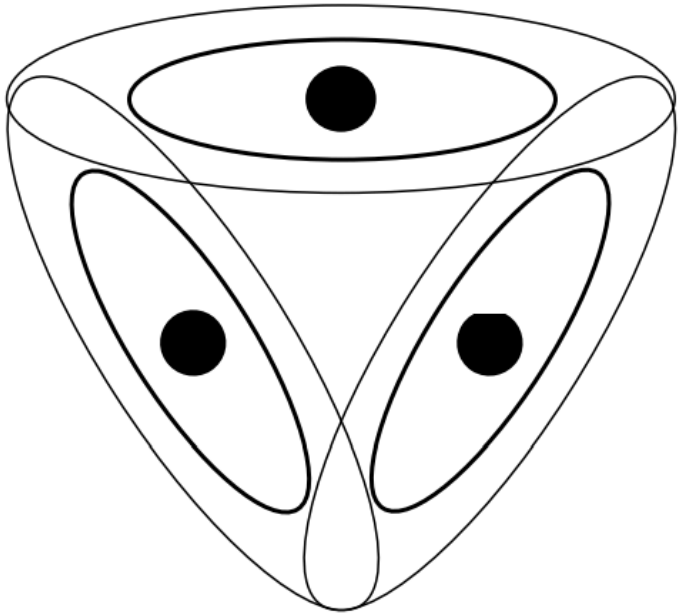
Distributed Constraint Satisfaction

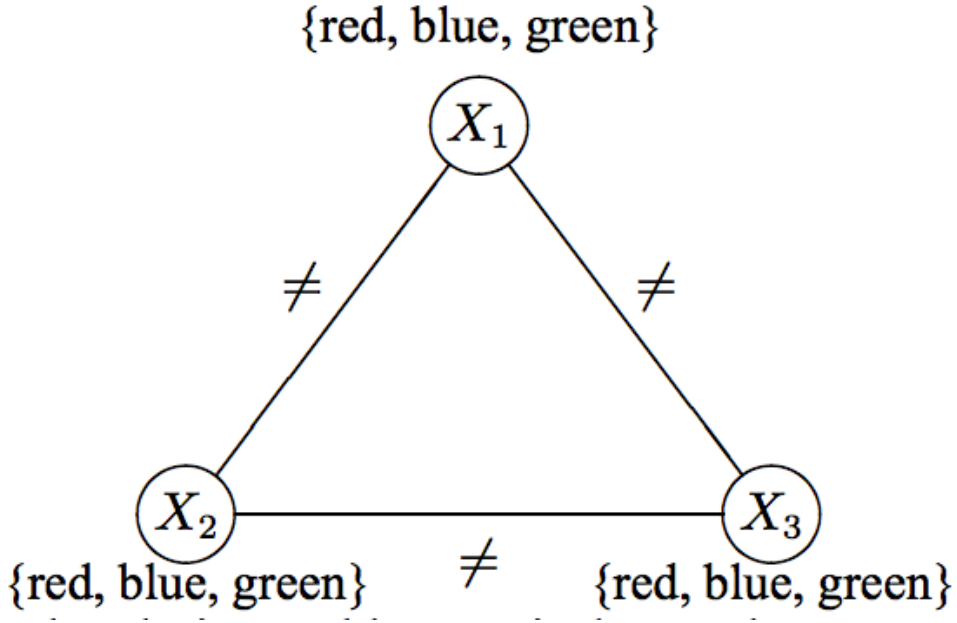
- Constraint Satisfaction Problem
  - Variables
  - Domains (per variable)
  - Constraints
- Solution: Legal instantiation of variables (i.e. no constraints are violated)
- There may be many solutions
- Assumptions:
  - Binary constraints
  - One solution suffices

Distributed Constraint Satisfaction

Distributed CSP: Each variable is owned by a single agent

No agent has a global view of the problem





Distributed Constraint Satisfaction

Distributed CSP: Each variable is owned by a single agent

- No agent has a global view of the problem

- Distributed algorithm: Each agent engages in a protocol for combining local computation with neighbors

Distributed Constraint Satisfaction

Distributed CSP: Each variable is owned by a single agent

We discuss two types of algorithms:

- Embodying a **least-commitment approach** and attempt to rule out impossible variable values without losing any possible solutions.

- Embodying a more adventurous spirit and **select tentative variable values,** backtracking when those choices prove unsuccessful.

Distributed Constraint Satisfaction

Distributed CSP: Each variable is owned by a single agent

Assumptions:

- the communication between neighboring nodes is perfect,
- messages can take more or less time without rhyme or reason
- If node i sends multiple messages to node j, those messages arrive in the order in which they were sent.

Distributed Constraint Satisfaction

Domain Pruning Algorithms

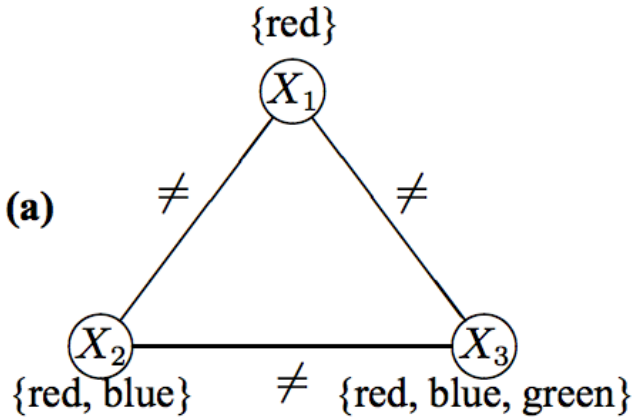
Basic idea: agents communicate with their neighbors in order to eliminate values from their domains

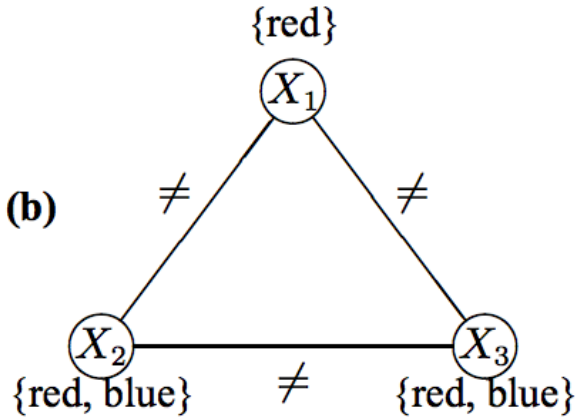
Filtering algorithm: each node communicates its domain to its neighbors, **maintains arc consistency** (i.e. eliminates from its domain the values that are not consistent with the values received from the neighbors), and the process repeats.

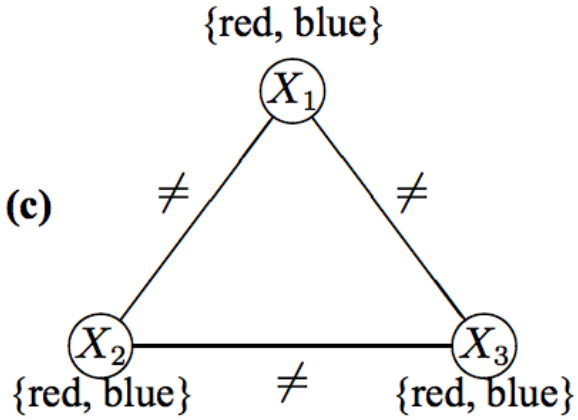
procedure Revise(x_i, x_j)
forall $v_i \in D_i$ **do**
  if *there is no value* $v_j \in D_j$ *such that* v_i *is consistent with* v_j **then**
    delete v_i from D_i

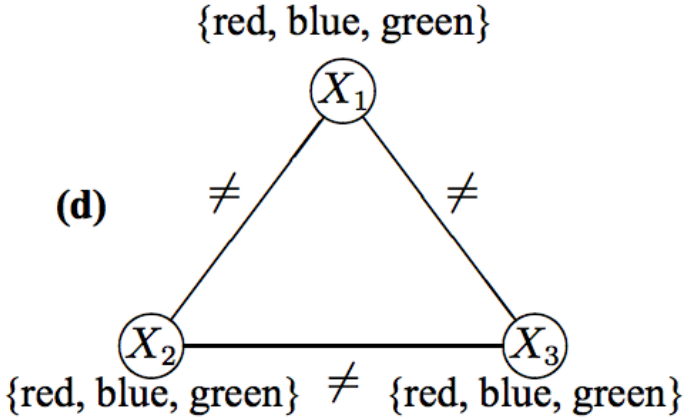
Sound but not complete

Distributed Constraint Satisfaction









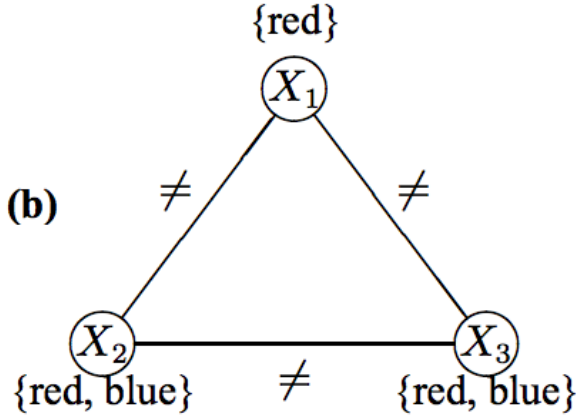
Distributed Constraint Satisfaction

Filtering algorithm with Unit Resolution

The algorithm is directly based on the **unit resolution** notion of unit resolution from propositional logic !

$$\frac{A_1 \quad \neg(A_1 \wedge A_2 \wedge \cdots \wedge A_n)}{\neg(A_2 \wedge \cdots \wedge A_n)}$$

Nogoods: Nogood{x_1 =red, x_2 =red}
equiv. $\neg(x_1 = red \wedge x_2 = red)$



$$\frac{x_1 = red \quad \neg(x_1 = red \wedge x_2 = red)}{\neg(x_2 = red)}$$

Distributed Constraint Satisfaction

Filtering algorithm with Hyper-Resolution

Hyper-resolution

$$\frac{\begin{array}{l} A_1 \vee A_2 \vee \cdots \vee A_m \\ \neg(A_1 \wedge A_{1,1} \wedge A_{1,2} \wedge \cdots) \\ \neg(A_2 \wedge A_{2,1} \wedge A_{2,2} \wedge \cdots) \\ \vdots \\ \neg(A_m \wedge A_{m,1} \wedge A_{m,2} \wedge \cdots) \end{array}}{\neg(A_{1,1} \wedge \cdots \wedge A_{2,1} \wedge \cdots \wedge A_{m,1} \wedge \cdots)}$$

Each agent repeatedly generates new constraints for his neighbors, notifies them of these new constraints, and prunes his own domain based on new constraints passed to him by his neighbors

Distributed Constraint Satisfaction

Filtering algorithm with Hyper-Resolution

Each agent repeatedly generates new constraints for his neighbors, notifies them of these new constraints, and prunes his own domain based on new constraints passed to him by his neighbors

procedure ReviseHR(NG_i, NG_j^*)

repeat

  $NG_i \leftarrow NG_i \bigcup NG_j^*$

  let NG_i^* denote the set of new Nogoods that i can derive from NG_i and his domain using hyper-resolution

  if NG_i^* *is nonempty* **then**

    $NG_i \leftarrow NG_i \bigcup NG_i^*$

    send the Nogoods NG_i^* to all neighbors of i

    if $\{\} \in NG_i^*$ **then**

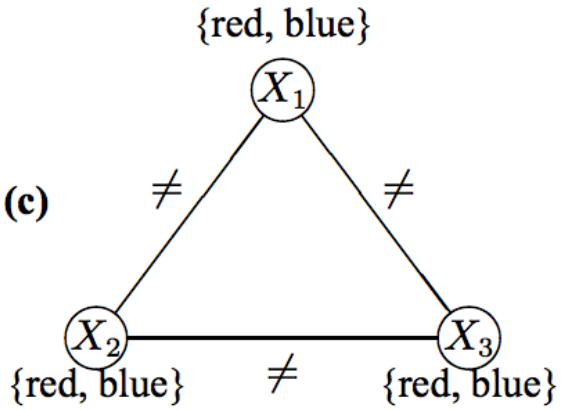
      stop

until *there is no change in* i*'s set of Nogoods* NG_i

Sound and complete

Distributed Constraint Satisfaction

Filtering algorithm with Hyper-Resolution



$$\frac{\begin{array}{l} x_1 = red \vee x_1 = blue \\ \neg(x_1 = red \wedge x_2 = red) \\ \neg(x_1 = blue \wedge x_3 = blue) \end{array}}{\neg(x_2 = red \wedge x_3 = blue)} \qquad \frac{\begin{array}{l} x_2 = red \vee x_2 = blue \\ \neg(x_2 = red \wedge x_3 = blue) \\ \neg(x_2 = blue \wedge x_3 = blue) \end{array}}{\neg(x_3 = blue)}$$

x_1 Nogoods

$\{x_1 = blue, x_2 = blue\}$

$\{x_1 = red, x_2 = red\}$

$\{x_1 = red, x_3 = red\}$

$\{x_1 = blue, x_3 = blue\}$

Additional x_1 Nogoods

$\{x_2 = red, x_3 = blue\}$

$\{x_2 = blue, x_3 = red\}$

Additional x_2 Nogoods

$(x_3 = blue)$

$\{x_3 = red\}$

Additional x_3 Nogoods

{}

Distributed Constraint Satisfaction

Filtering algorithm:

- The number of Nogoods can grow to be unmanageably large
- Problem: Least commitment

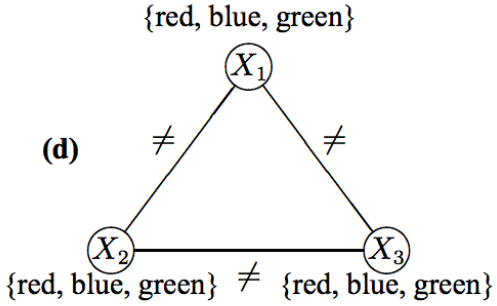
Alternative:
Make tentative value selections for variables and backtrack

Distributed Constraint Satisfaction

Heuristic search algorithms:
Make tentative value selections for variables and backtrack

No distribution (complete)

procedure ChooseValue(x_i, $\{v_1, v_2, \ldots, v_{i-1}\}$)
$v_i \leftarrow$ value from the domain of x_i that is consistent with $\{v_1, v_2, \ldots, v_{i-1}\}$
if *no such value exists* **then**
  backtrack[1]
else if $i = n$ **then**
  stop
else
  ChooseValue(x_{i+1}, $\{v_1, v_2, \ldots, v_i\}$)



select a value from your domain
repeat
  if *your current value is consistent with the current values of your neighbors, or if none of the values in your domain are consistent with them* **then**
    do nothing
  else
    select a value in your domain that is consistent with those of your neighbors and notify your neighbors of your new value
until *there is no change in your value*

Distributed Constraint Satisfaction

Thus, parallelism, asynchrony, soundness and completeness need to be reconciled.

Asynchronous backtracking

- Total ordering of agents (prioritizing agents)
- Message-passing protocol (based on ordering)

Distributed Constraint Satisfaction

Asynchronous backtracking

- Each binary constraint is known to both the constrained agents and is checked in the algorithm by the agent with the lower priority between the two.
- A link in the constraint network is always directed from an agent with higher priority to an agent with lower priority

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

1. Agents instantiate their variables concurrently
2. Send their assigned values to the agents that are connected to them by outgoing links. All agents wait for and respond to messages.
3. After each update of his assignment, an agent sends his new assignment along all outgoing links.
4. An agent who receives an assignment (from the higher-priority agent of the link), tries to find an assignment for his variable that does not violate a constraint with the assignment he received.

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

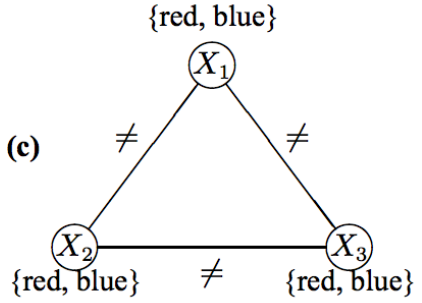
when *received (**Ok?**, (A_j, d_j))* **do**
- add (A_j, d_j) to *agent_view*
- **check_agent_view**

when *received (**Nogood**,* nogood*)* **do**
- add *nogood* to Nogood list
- **forall** (A_k, d_k) ∈ nogood, *if A_k is not a neighbor of A_i* **do**
  - add (A_k, d_k) to *agent_view*
  - request A_k to add A_i as a neighbor
- **check_agent_view**

procedure check_agent_view
when agent_view *and* current_value *are inconsistent* **do**
- **if** *no value in D_i is consistent with* agent_view **then**
  - **backtrack**
- **else**
  - select $d \in D_i$ consistent with *agent_view*
  - *current_value* ← d
  - send (**ok?**, (A_i, d)) to lower-priority neighbors

procedure backtrack
nogood ← some inconsistent set, using hyper-resolution or similar procedure
if nogood *is the empty set* **then**
- broadcast to other agents that there is no solution
- terminate this algorithm

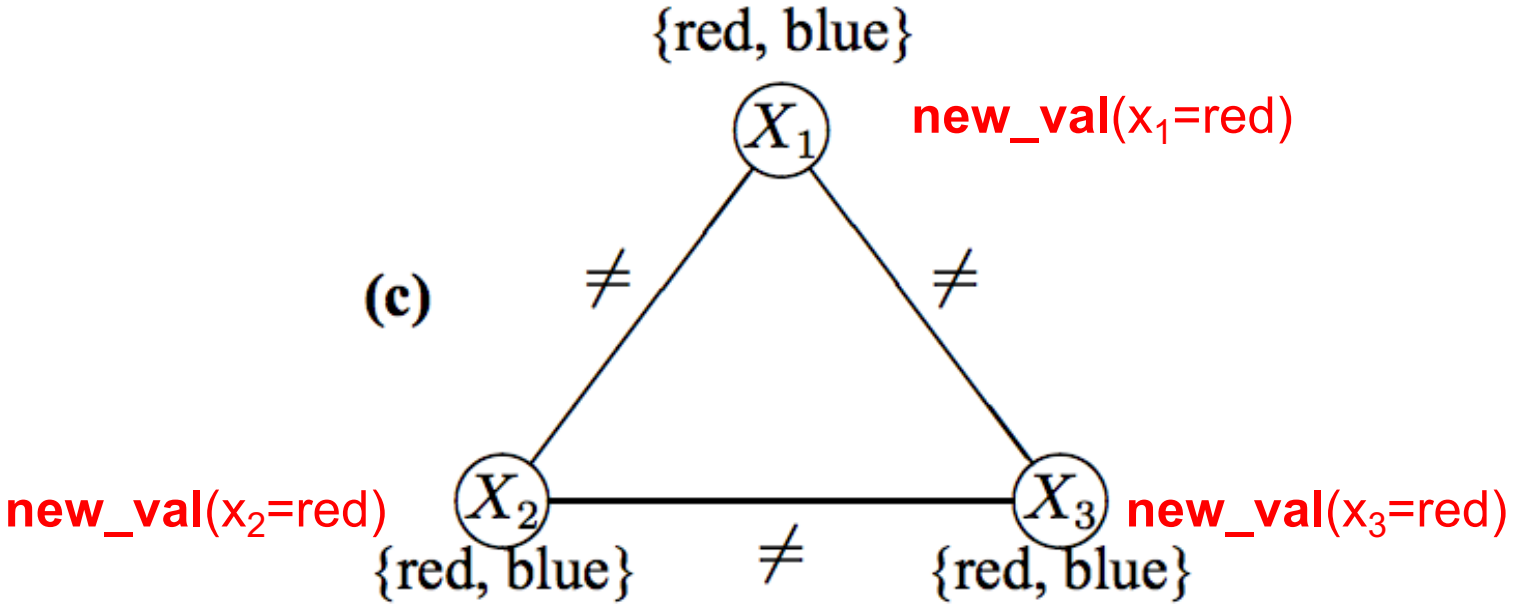
else
- select $(A_j, d_j) \in$ *nogood* where A_j has the lowest priority in *nogood*
- send (**Nogood**, *nogood*) to A_j
- remove (A_j, d_j) from *agent_view*
- **check_agent_view**



Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

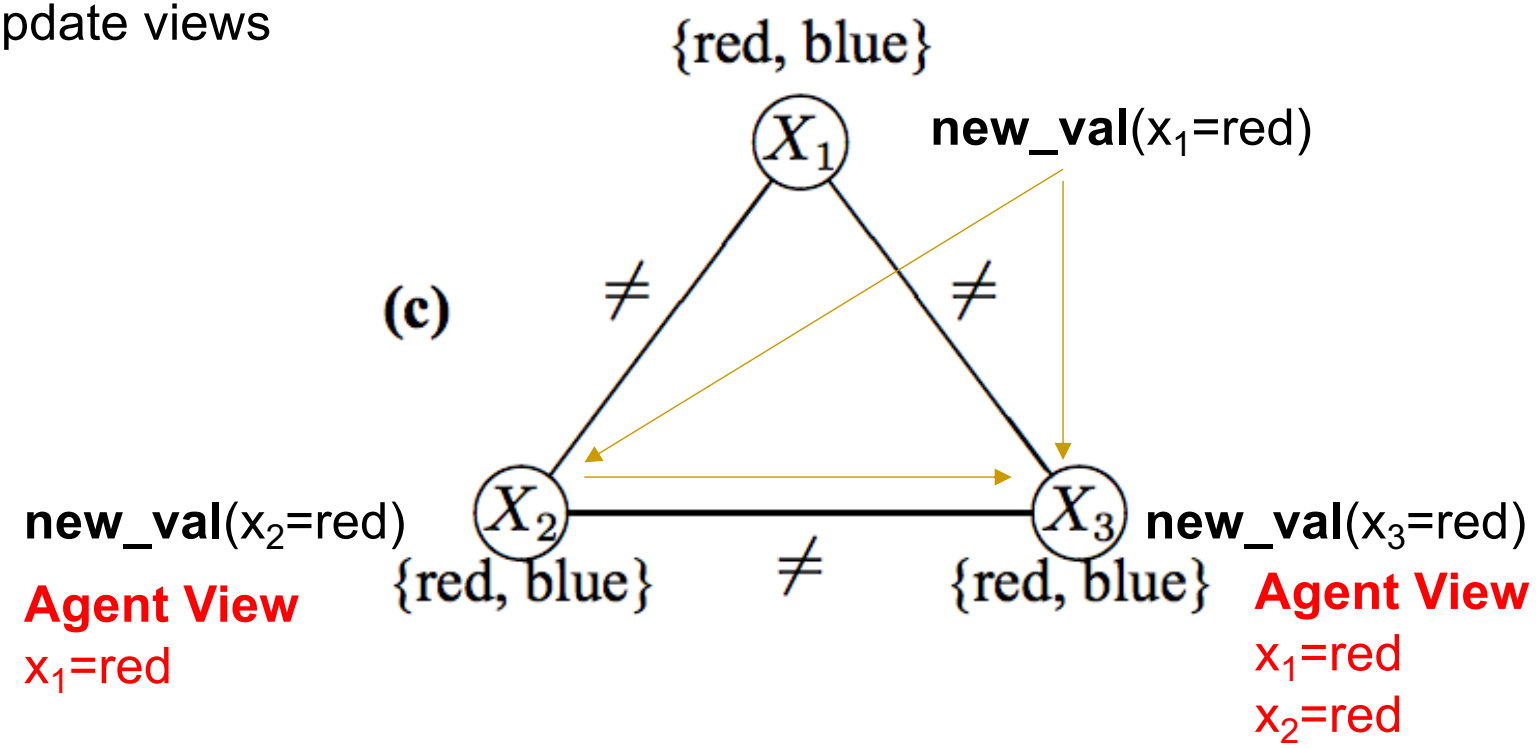
Priority: X1, X2, X3
Step: all take values



Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

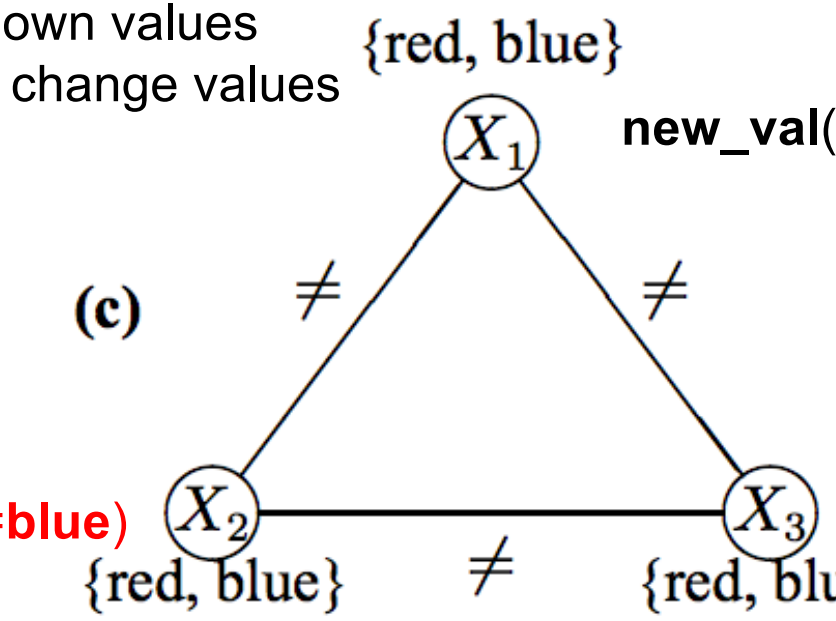
Priority: X1, X2, X3
Step: communicate values & update views



Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X2 and X3 check consistency of own values with views and change values



new_val(x_1=red)

new_val(x_2=blue)

Agent View
x_1=red

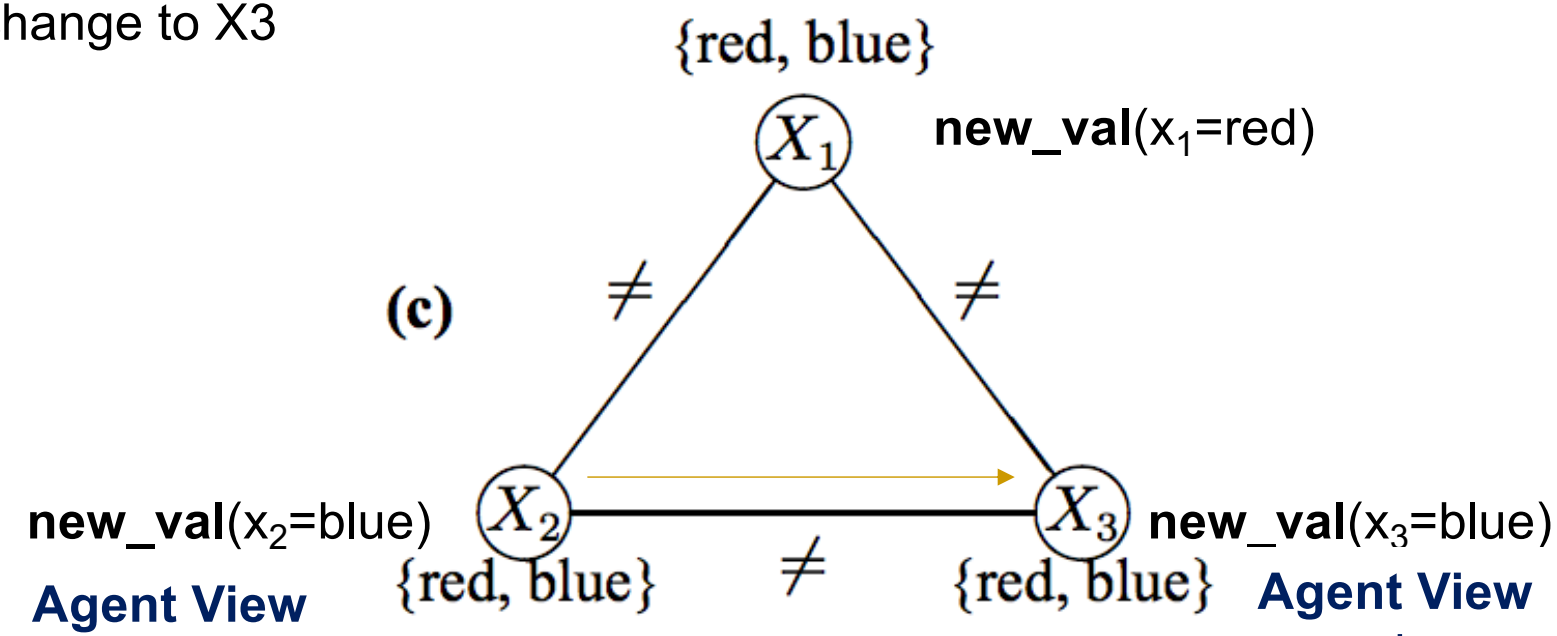
new_val(x_3=blue)

Agent View
x_1=red
x_2=red

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

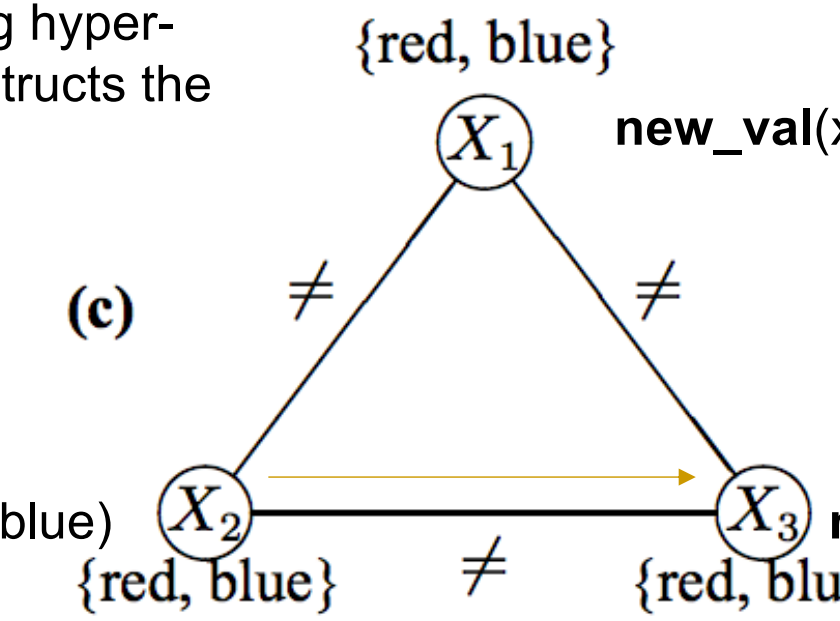
Priority: X1, X2, X3
Step: X2 communicates change to X3



Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X3 cannot find own value and using hyper-resolution constructs the NoGood list



new_val(x_1=red)

new_val(x_2=blue)

Agent View
x_1=red

new_val(x_3=blue)

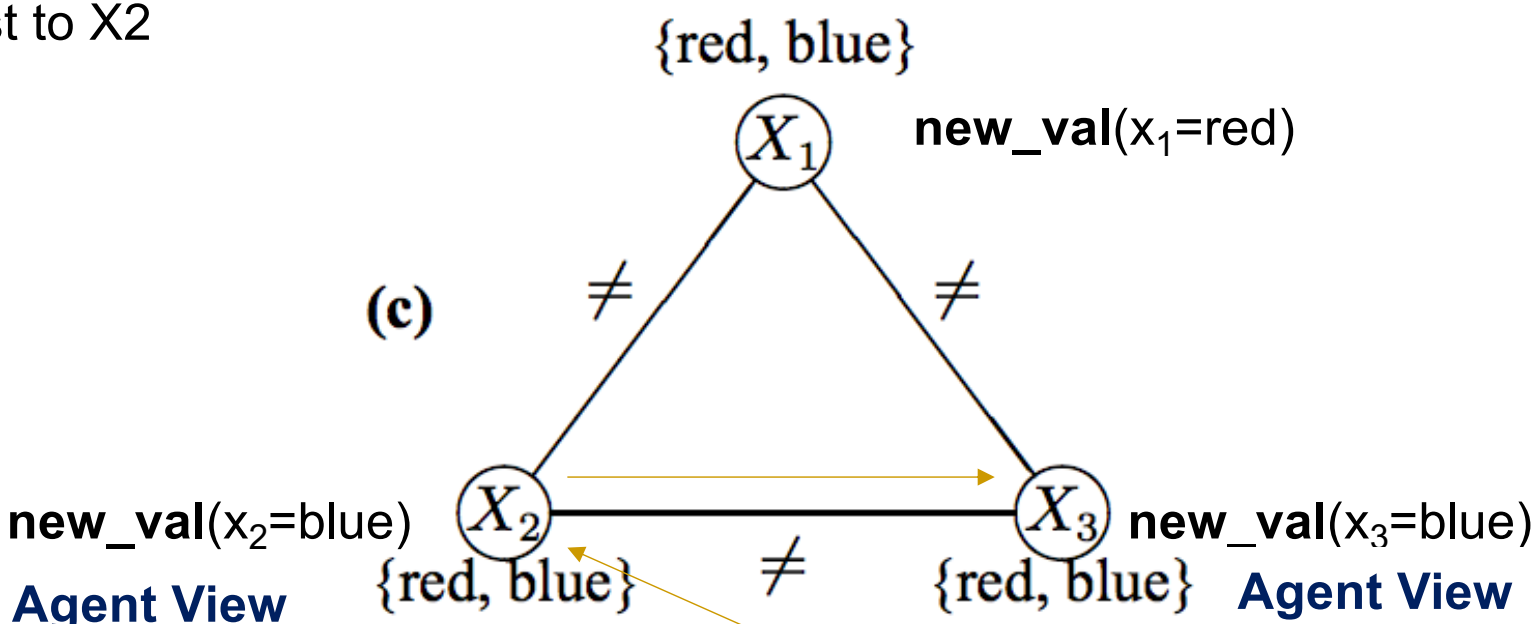
Agent View
x_1=red
x_2=blue

NoGood
x_1=red
x_2=blue

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

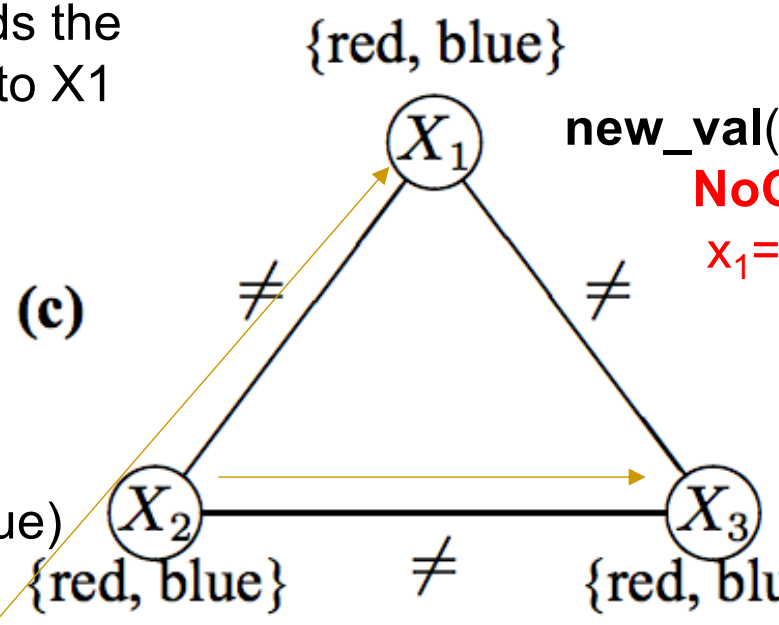
Priority: X1, X2, X3
Step: X3 sends the NoGood list to X2



Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X2 cannot find a value for itself and sends the NoGood{x_1=red} to X1



new_val(x_1=red)
NoGood
x_1=red

new_val(x_2=blue)
Agent View
x_1=red

NoGood
x_1=red
x_2=blue

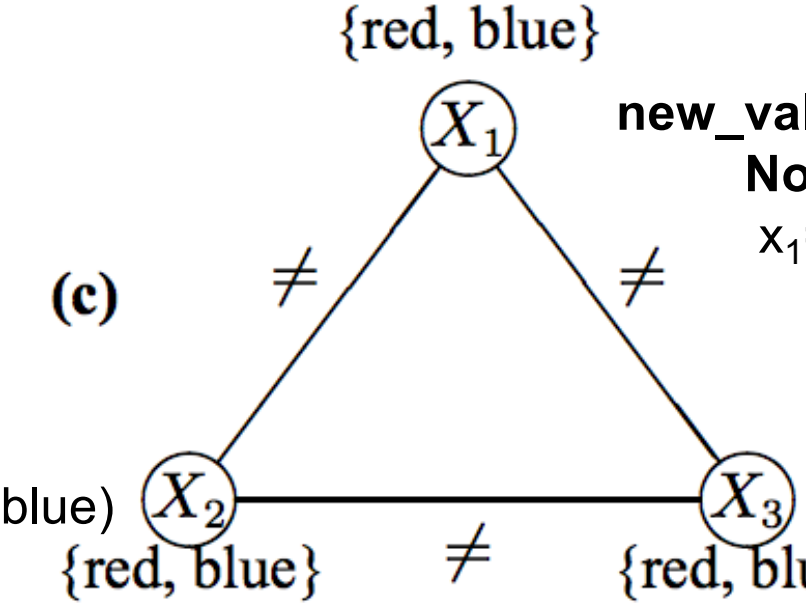
new_val(x_3=blue)
Agent View
x_1=red

NoGood
x_1=red
x_2=blue

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X1 changes its own value to blue



new_val(x_1=blue)
NoGood
x_1=red

new_val(x_2=blue)
Agent View

NoGood
x_1=red
x_2=blue

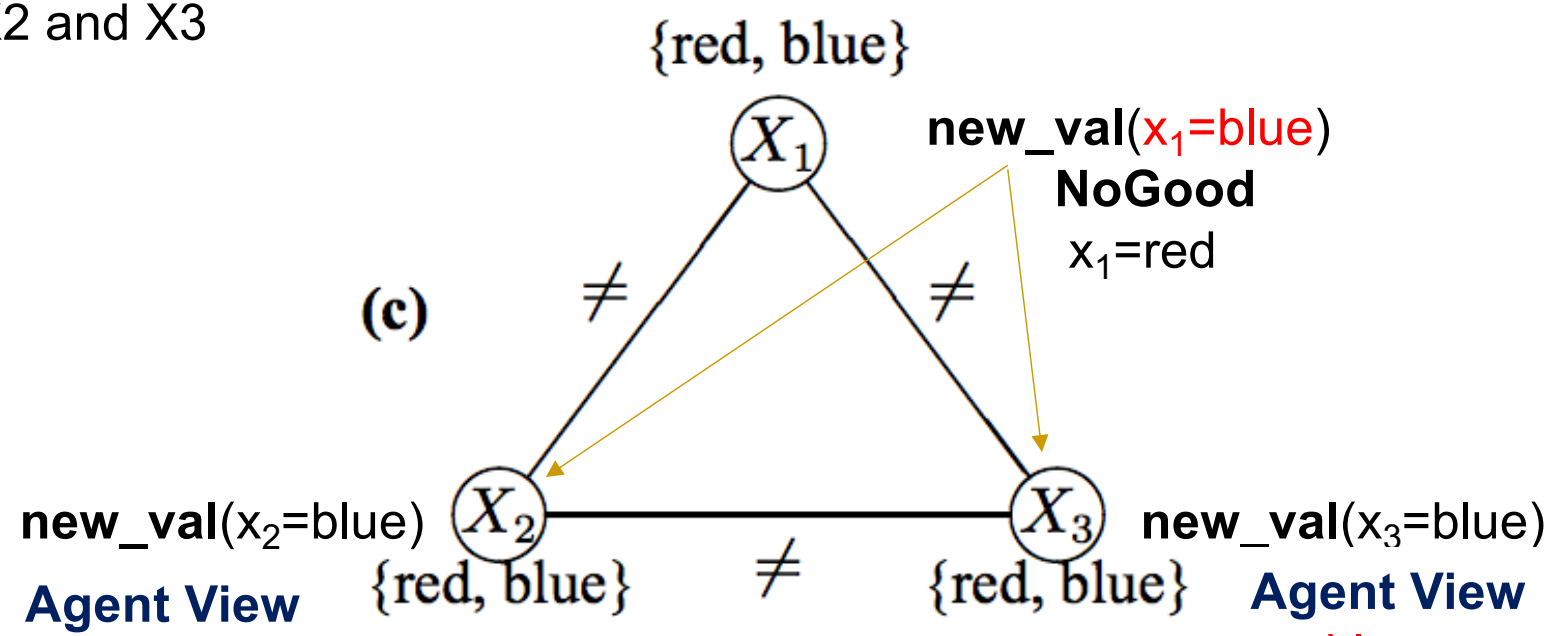
new_val(x_3=blue)
Agent View
x_1=red

NoGood
x_1=red
x_2=blue

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

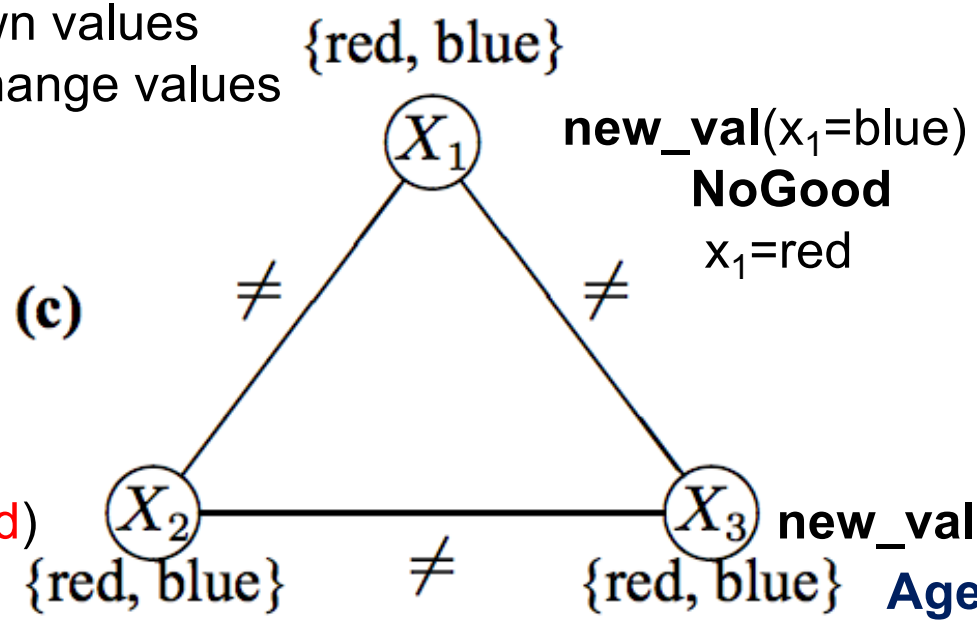
Priority: X1, X2, X3
Step: X1 sends the update to X2 and X3



Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X2 and X3 check consistency of own values with views and change values



new_val(x_1=blue)
NoGood
x_1=red

new_val(x_2=red)
Agent View
x_1=blue

NoGood
x_1=red
x_2=blue

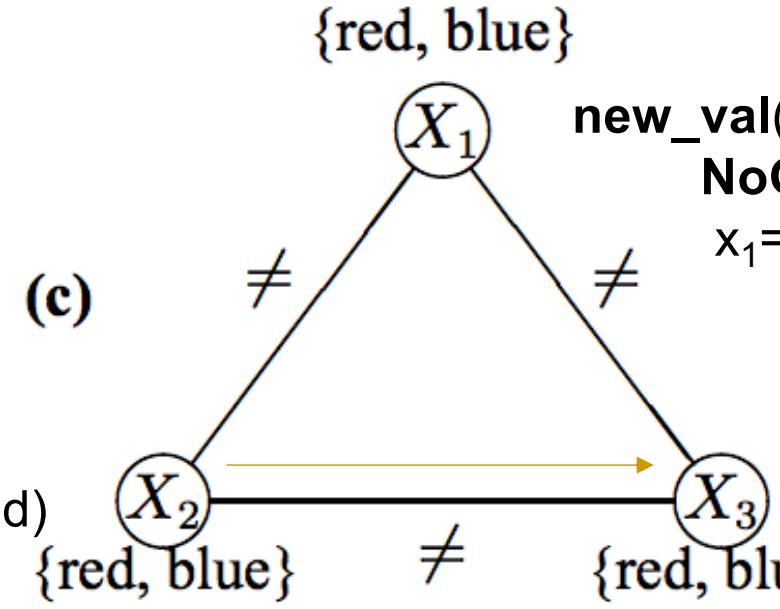
new_val(x_3= red)
Agent View
x_1=blue

NoGood
x_1=red
x_2=blue

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X2 communicates change to X3



new_val(x_1=blue)
NoGood
x_1=red

new_val(x_2=red)
Agent View
x_1=blue
NoGood
x_1=red
x_2=blue

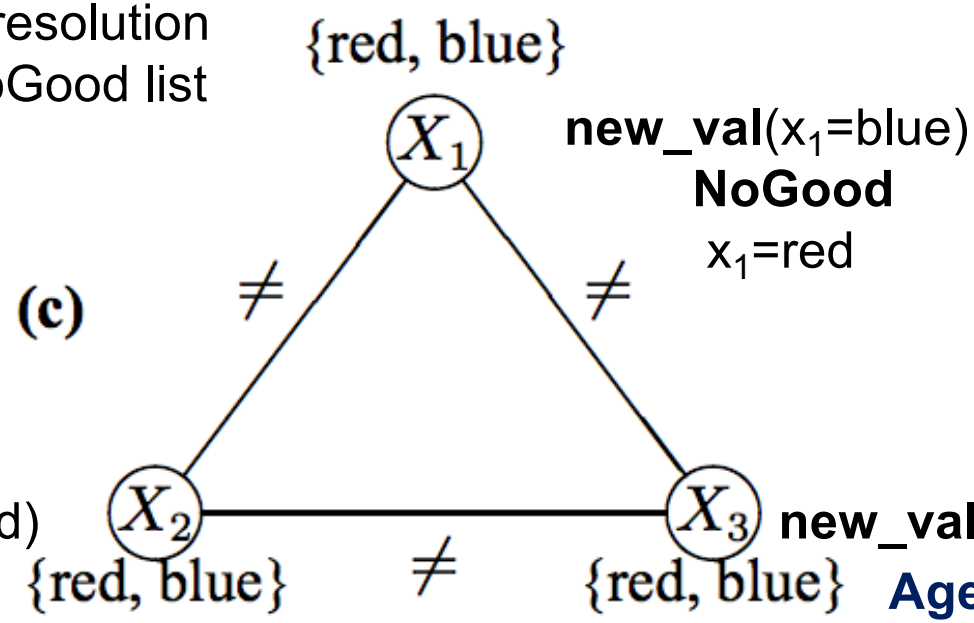
new_val(x_3= red)
Agent View
x_1=blue
x_2=red

NoGood
x_1=red
x_2=blue

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X3 cannot find value and using hyper-resolution constructs the NoGood list



new_val(x_1=blue)
NoGood
x_1=red

new_val(x_2=red)
Agent View
x_1=blue
NoGood
x_1=red
x_2=blue

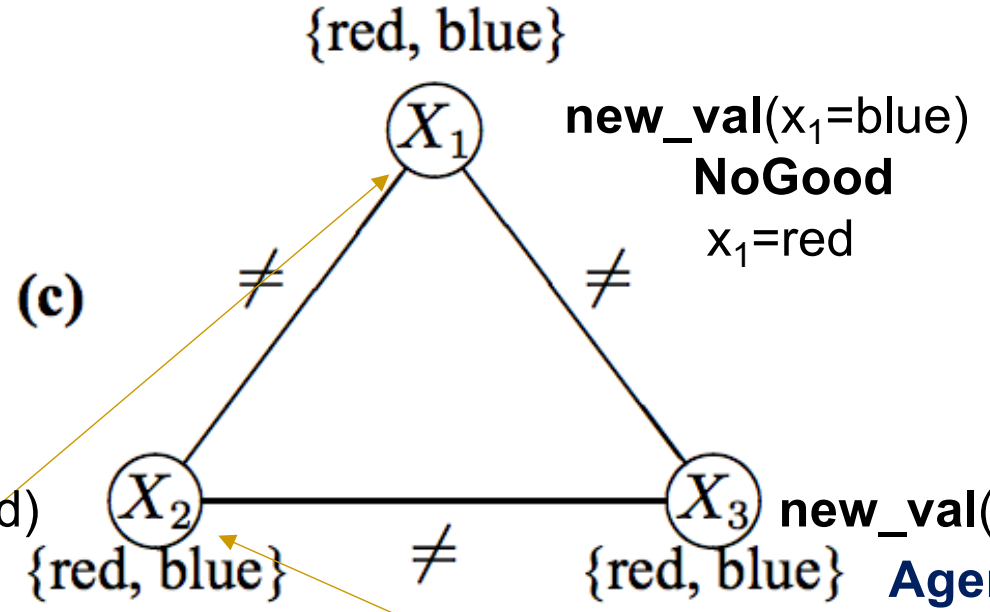
new_val(x_3= red)
Agent View
x_1=blue
x_2=red

NoGood
x_1=blue
x_2=red

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X3 sends the NoGood list to X2



new_val(x_1=blue)
NoGood
x_1=red

new_val(x_2=red)
Agent View
x_1=blue
NoGood
x_1=red
x_2=blue
x_1=blue
x_2=red

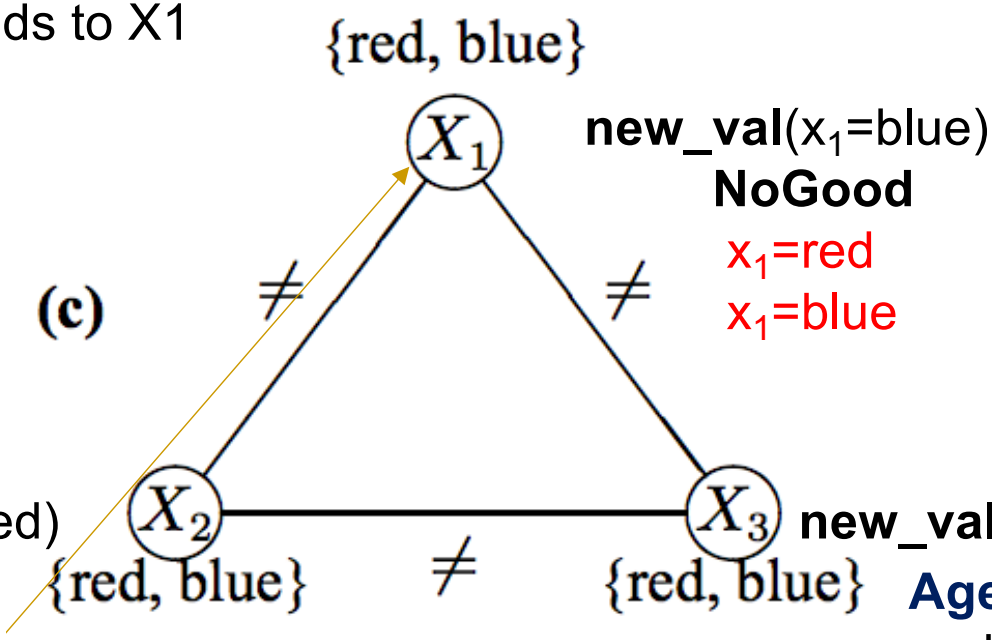
new_val(x_3= red)
Agent View
x_1=blue
x_2=red

NoGood
x_1=blue
x_2=red

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X2 cannot find a value for itself and sends to X1



(c)

new_val(x_1=blue)
NoGood
x_1=red
x_1=blue

new_val(x_2=red)
Agent View
x_1=blue
NoGood
x_1=red
x_2=blue
x_1=blue
x_2=red

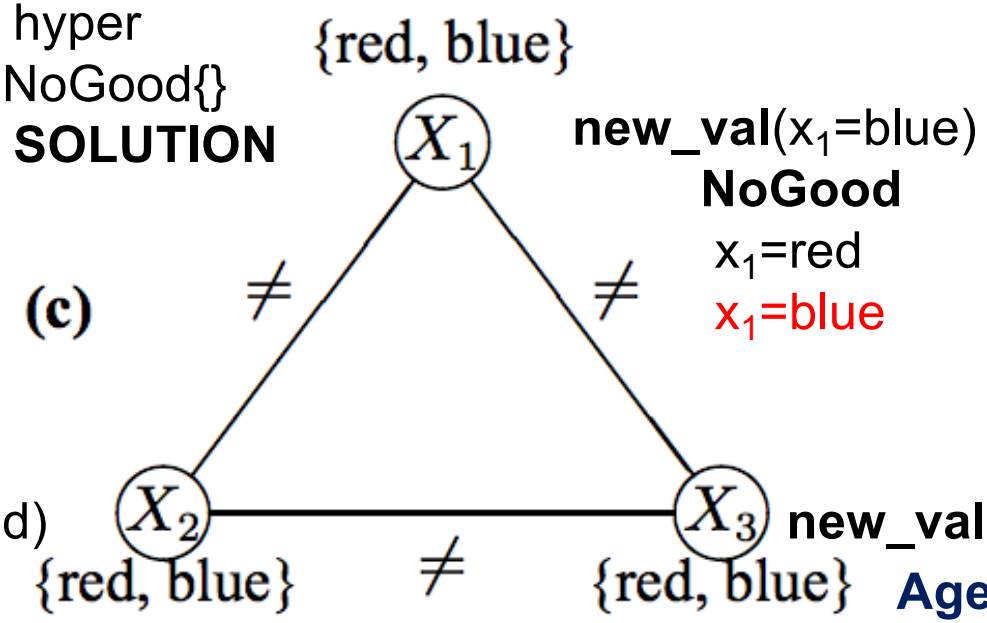
new_val(x_3= red)
Agent View
x_1=blue
x_2=red

NoGood
x_1=blue
x_2=red

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

Priority: X1, X2, X3
Step: X1 has NoGood for both values and using hyper resolution it gets NoGood{} and declares **NO SOLUTION**



new_val(x_1=blue)
NoGood
x_1=red
x_1=blue

new_val(x_2=red)
Agent View
x_1=blue
NoGood
x_1=red
x_2=blue
x_1=blue
x_2=red

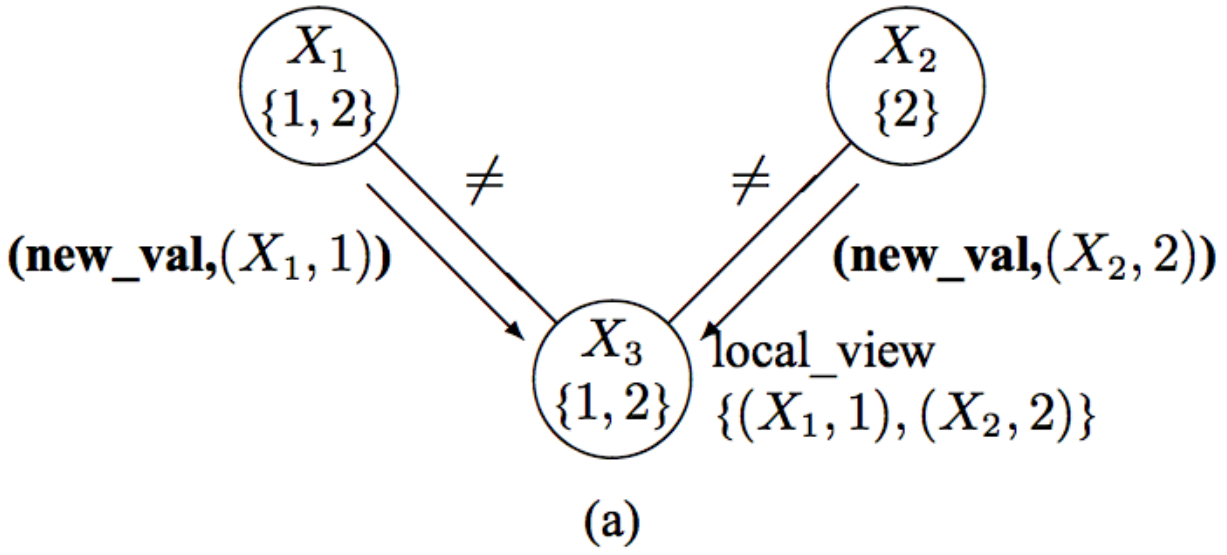
new_val(x_3= red)
Agent View
x_1=blue
x_2=red

NoGood
x_1=blue
x_2=red

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

With dynamic link addition

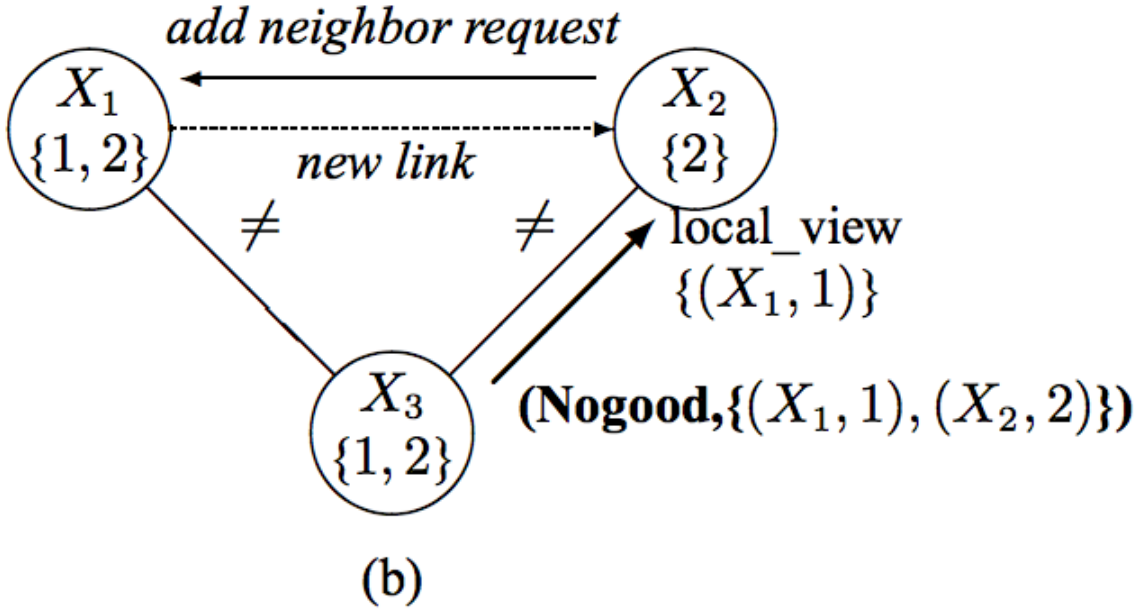


(a)

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

With dynamic link addition

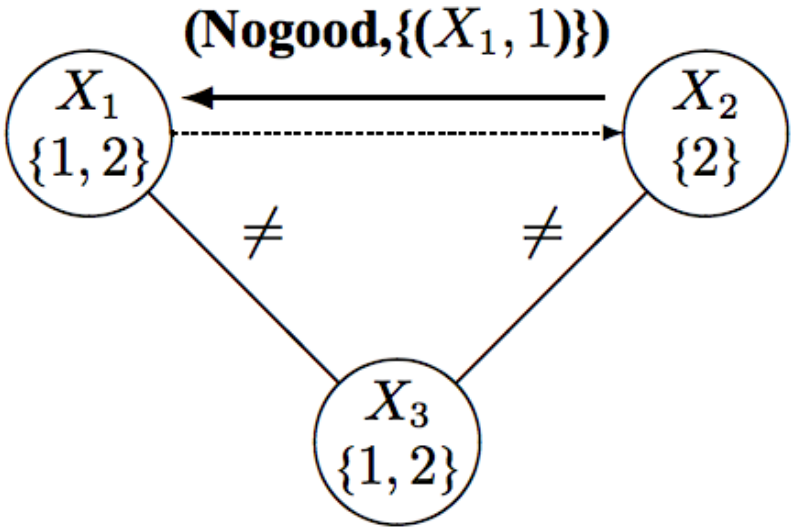


(b)

Distributed Constraint Satisfaction

Asynchronous BackTracking (ABT)

With dynamic link addition



Distributed Optimization

Question: how agents can, in a distributed fashion, optimize a global objective function.

Four **families of techniques**:

- Distributed dynamic programming (as applied to path-planning problems).
- Optimization algorithms with an economic flavor (as applied to matching and scheduling problems).
- Coordination via social laws and conventions, and the example of traffic rules.
- Distributed solutions to Markov Decision Problems (MDPs) (not discussed here)

Distributed Optimization

Distributed dynamic programming for path planning

Like graph coloring, path planning constitutes another common abstract problem-solving framework. A path-planning problem consists of a weighted directed graph with a set of n nodes N, directed links L, a weight function $w : L \mapsto \mathbb{R}^+$, and two nodes $s, t \in N$. The goal is to find a directed path from s to t having minimal total weight. More generally, we consider a set of goal nodes $T \subset N$, and are interested in the shortest path from s to any of the goal nodes $t \in T$.

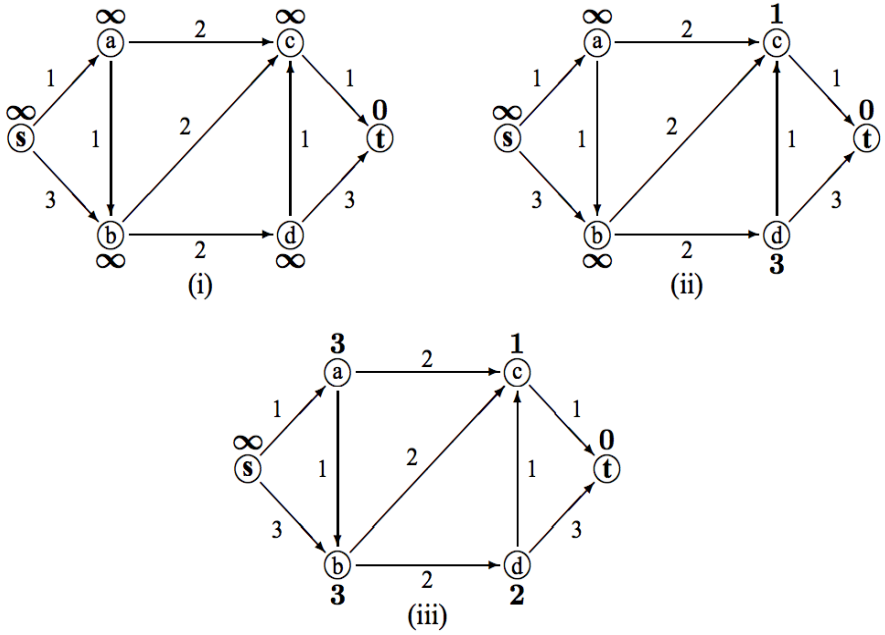
Principle of optimality: if a node x lies on a shortest path from s optimality to t, then the portion of the path from s to x (or, respectively, from x to t) must also be the shortest paths between s and x (resp., x and t).

This allows an incremental dynamic divide-and-conquer procedure, also known as dynamic programming.

Distributed Optimization

Distributed dynamic programming for path planning

procedure ASYNCHDP (node i)
if i *is a goal node* **then**
  $h(i) \leftarrow 0$
else
  initialize $h(i)$ arbitrarily (e.g., to ∞ or 0)
repeat
  forall *neighbors* j **do**
    $f(j) \leftarrow w(i,j) + h(j)$
  $h(i) \leftarrow \min_j f(j)$



Guaranteed to converge BUT Not realistic !

Distributed Optimization

Learning Real-Time A* (LRTA*)

(single agent with admissible heuristic function h())

procedure LRTA*
$i \leftarrow s$ // the start node
while *i is not a goal node* **do**
  foreach *neighbor* j **do**
    $f(j) \leftarrow w(i,j) + h(j)$
  $i' \leftarrow \arg\min_j f(j)$ // breaking ties at random
  $h(i) \leftarrow \max(h(i), f(i'))$
  $i \leftarrow i'$

Distributed Optimization

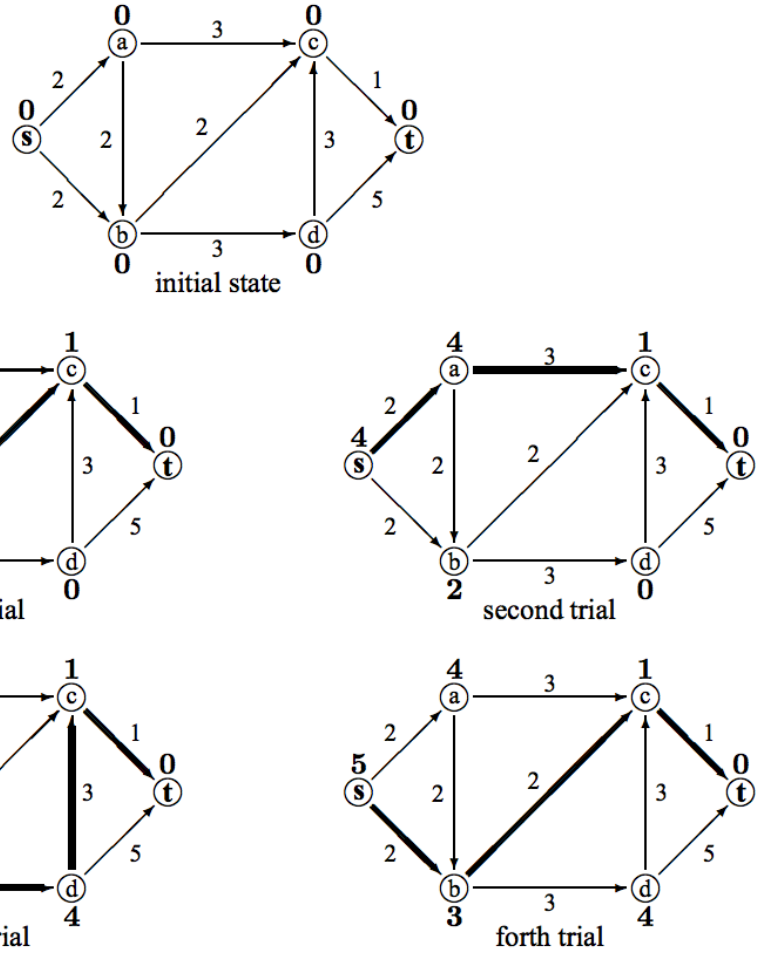
Learning Real-Time A* (LRTA*)

(single agent with admissible heuristic function h())

- The h-values never decrease, and remain admissible.
- LRTA∗ terminates; the complete execution from the start node to termination at the goal node is called a trial.
- If LRTA∗ is repeated while maintaining the h-values from one trial to the next, it eventually discovers the shortest path from the start to a goal node.
- If LRTA∗ finds the same path on two sequential trials, this is the shortest path. (However, this path may also be found in one or more previous trials before it is found twice in a row. Do you see why?)

Distributed Optimization

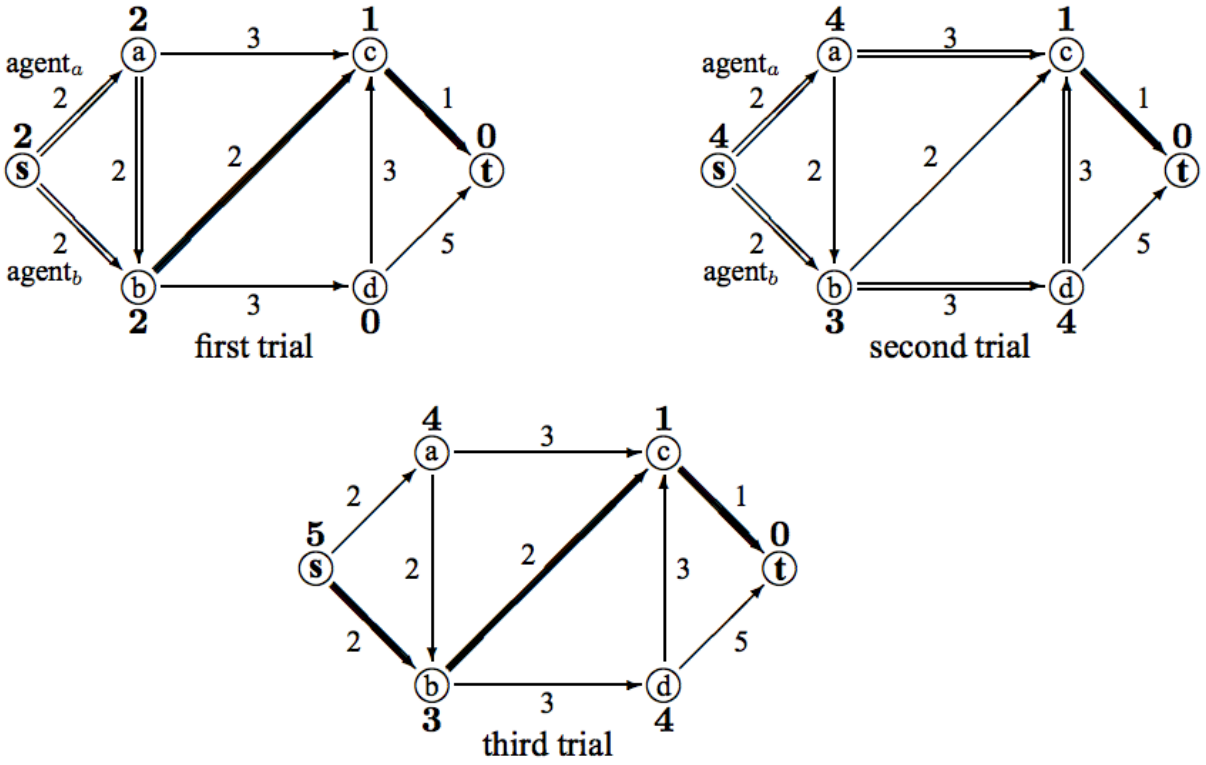
Learning Real-Time A* (LRTA*)



Distributed Optimization

Learning Real-Time A* (n) (LRTA*(n))

Here for 2 agents i.e. LRTA*(2)



Distributed Optimization

Contract nets and auctions

- A TOD is a triple

$$<T, Ag, c>$$

where

  - T is the (finite) set of all possible tasks
  - $Ag = \{1,...,n\}$ is the set of participating agents
  - $c = \wp(T) \rightarrow$ |R defines the *cost* of executing each subset of tasks

- An *encounter* is a collection of tasks

$$<T_1,...,T_n>$$

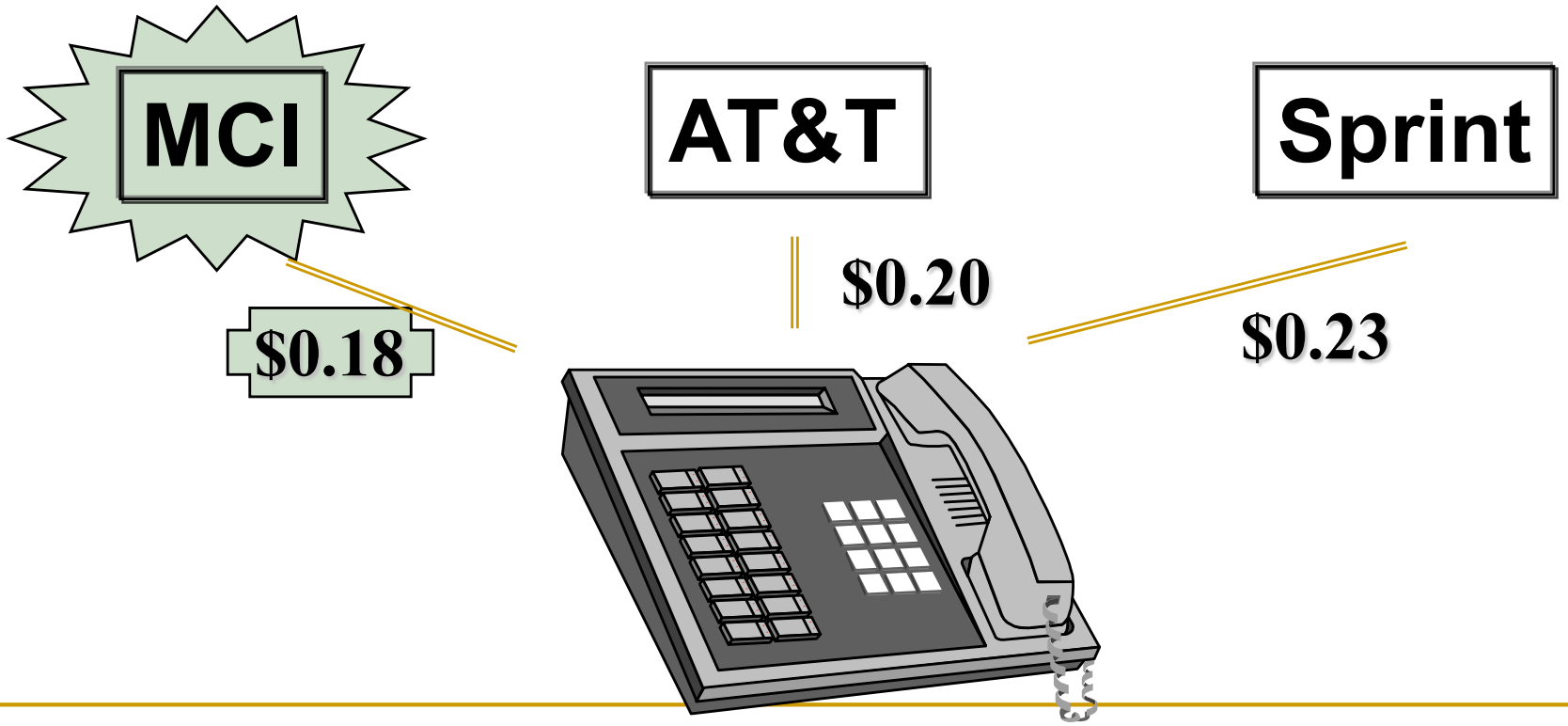
where $T_i \subseteq T$ for each $i \in Ag$

Phone Call Competition Example

- Customer wishes to place long-distance call
- Carriers simultaneously bid, sending proposed prices
- Phone automatically chooses the carrier (dynamically)

Best Bid Wins

- Phone chooses carrier with lowest bid
- Carrier gets amount that it bid



Distributed Optimization

Contract nets and auctions

The Contract Net Protocol (CNET)

• Protocol for task sharing (problem distribution) among communicating problem solvers (→ agents); basic modelling metaphor: contracts.

• Primary concerns: distributed control, achieving reliability, avoiding bottlenecks

• Stages in CNET:

1) (Recognition)
2) Announcement
3) Bidding
4) Awarding
5) Expediting

Distributed Optimization

Contract nets and auctions

The Contract Net Protocol (CNET) -- Features

- Two-way transfer of information
- Local Evaluation
- Mutual selection (bidders select from among task announcements, managers select from among bids)

CNET vs. other mechanisms for transfer of control

- CNET views transfer of control as runtime, symmetric process involving transfer of complex information in order to be effective
- Other mechanisms (procedure invocation, production rules, pattern directed invocation, blackboards) are unidirectional, minimally run-time sensitive, and have restricted communication

Distributed Optimization

Contract nets and auctions

The Contract Net Protocol (CNET) – Limitations (not drawbacks)

- Before sub-problems can be distributed (→ announcements can be made), problem decomposition needs to be performed (highly non-trivial)
- communication produces overhead, is slow
- problems must have right granularity (rather coarse)
- recognition stage (agent realizes that it needs help with a problem) is not explicitly covered

Distributed Optimization

Contract nets and auctions

Questions

1. We start with some global problem to be solved, but then speak about minimizing the total cost to the agents. What is the connection between the two?

Ans: In certain settings an optimal solution is closely related to the utilities of agents (solution concepts in game theory)

Distributed Optimization

Contract nets and auctions

Questions

1. We start with some global problem to be solved, but then speak about minimizing the total cost to the agents. What is the connection between the two?
2. When exactly do agents make offers, and what is the precise method by which the contracts are decided on?

Ans: Auctions provide an answer. Every negotiation scheme bidding rule consists of three elements: (1) permissible ways of making offers (bidding rules), (2) definition of the outcome based on the offers (market clearing rules), and (3) market clearing rule the information made available to the agents throughout the process (information dissemination rules).

Distributed Optimization

Contract nets and auctions

Questions

1. We start with some global problem to be solved, but then speak about minimizing the total cost to the agents. What is the connection between the two?
2. When exactly do agents make offers, and what is the precise method by which the contracts are decided on?
3. Since we are in a cooperative setting, why does it matter whether agents “lose money" or not on a given contract?

Ans: In the spirit of the contract-net paradigm, our auctions agents will engage in a series of bids for resources, and at the end of the auction the assignment of the resources to the “winners” of the auction will constitute an optimal (or near optimal, in some cases) solution.

In the standard treatment of auctions (to be discussed) the bidders are assumed to bid in a way that maximizes their personal payoff.

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem

A (symmetric) assignment problem *consists of*

- *A set* N *of* n *agents,*
- *A set* X *of* n *objects,*
- *A set* $M \subseteq N \times X$ *of possible assignment pairs, and*
- *A function* $v : M \mapsto \mathbb{R}$ *giving the value of each assignment pair.*

An assignment is a set of pairs $S \subseteq M$ *such that each agent* $i \in N$ *and each object* $j \in X$ *is in at most one pair in* S. *A* feasible assignment *is one in which all agents are assigned an object. A feasible assignment* S *is* optimal *if it maximizes* $\sum_{(i,j)\in S} v(i,j)$.

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem

i	$v(i, x_1)$	$v(i, x_2)$	$v(i, x_3)$
1	2	4	0
2	1	5	0
3	1	3	2

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- As a linear program

$$\text{maximize} \quad \sum_{(i,j)\in M} v(i,j)x_{i,j}$$

$$\text{subject to} \quad \sum_{j|(i,j)\in M} x_{i,j} \leq 1 \qquad \forall i \in N$$

$$\sum_{i|(i,j)\in M} x_{i,j} \leq 1 \qquad \forall j \in X$$

BUT:
- **$O(n^3)$**
- **Not obviously parallelizable**
- **Not robust to changes**

Lemma *The LP encoding of the assignment problem has a solution such that for every i, j it is the case that $x_{i,j} = 0$ or $x_{i,j} = 1$. Furthermore, any optimal fractional solution can be converted in polynomial time to an optimal integral solution.*

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

Let $p = (p_1, \ldots, p_n)$, where p_j is the price of object j

Given an assignment *S* and the price vector, the "utility" of agent *i* *can be*

$$u(i,j) = v(i,j) - p_j$$

Definition **(Competitive equilibrium)** *A feasible assignment S and a price vector p are in* competitive equilibrium *when for every pairing $(i,j) \in S$ it is the case that $\forall k,\ u(i,j) \geq u(i,k)$.*

Theorem *If a feasible assignment S and a price vector p satisfy the competitive equilibrium condition then S is an optimal assignment. Furthermore, for any optimal solution S, there exists a price vector p such that p and S satisfy the competitive equilibrium condition.*

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem

Price vector : (2,4,1)

i	$v(i, x_1)$	$v(i, x_2)$	$v(i, x_3)$
1	2	4	0
2	1	5	0
3	1	3	2

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

Let $p = (p_1,$

Given an assig

HOW?

f agent j *can be*

Definition ...ent S *and a price vector* p *are in* competitive equilibrium *when for every pairing* $(i, j) \in S$ *it is the case that* $\forall k,\ u(i, j) \geq u(i, k)$.

Theorem *If a feasible assignment* S *and a price vector* p *satisfy the competitive equilibrium condition then* S *is an optimal assignment. Furthermore, for any optimal solution* S*, there exists a price vector* p *such that* p *and* S *satisfy the competitive equilibrium condition.*

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

How: Auction

Naive Auction Algorithm

// Initialization:

$S \leftarrow \emptyset$

forall $j \in X$ **do**

  $p_j \leftarrow 0$

repeat

  // Bidding Step:

  let $i \in N$ be an unassigned agent

  // Find an object $j \in X$ that offers i maximal value at current prices:

  $j \in \arg\max_{k|(i,k)\in M}(v(i,k) - p_k)$

  // Compute i's bid increment for j:

  $b_i \leftarrow (v(i,j) - p_j) - \max_{k|(i,k)\in M; k\neq j}(v(i,k) - p_k)$

  // which is the difference between the value to i of the best and second-best objects at current prices (note that i's bid will be the current price plus this bid increment).

  // Assignment Step:

  add the pair (i,j) to the assignment S

  if *there is another pair* (i',j) **then**

    remove it from the assignment S

  increase the price p_j by the increment b_i

until S *is feasible* // that is, it contains an assignment for all $i \in N$

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

round	p_1	p_2	p_3	bidder	preferred object	bid incr.	current assignment
0	0	0	0	1	x_2	2	$(1, x_2)$
1	0	2	0	2	x_2	2	$(2, x_2)$
2	0	4	0	3	x_3	1	$(2, x_2), (3, x_3)$
3	0	4	1	1	x_1	2	$(2, x_2), (3, x_3), (1, x_1)$

May NOT terminate

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

i	$v(i, x_1)$	$v(i, x_2)$	$v(i, x_3)$
1	1	1	0
2	1	1	0
3	1	1	0

round	p_1	p_2	p_3	bidder	preferred object	bid incr.	current assignment
0	0	0	0	1	x_1	0	$(1, x_1)$
1	0	0	0	2	x_2	0	$(1, x_1), (2, x_2)$
2	0	0	0	3	x_1	0	$(3, x_1), (2, x_2)$
3	0	0	0	1	x_2	0	$(3, x_1), (1, x_2)$
4	0	0	0	2	x_1	0	$(2, x_1), (1, x_2)$
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Solution: Add ϵ to the biding increment

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

Solution: Add ϵ to the biding increment

$$b_i = u(i,j) - \max_{k|(i,k)\in M; k\neq j} u(i,k) + \epsilon$$

i	$v(i,x_1)$	$v(i,x_2)$	$v(i,x_3)$
1	1	1	0
2	1	1	0
3	1	1	0

round	p_1	p_2	p_3	bidder	preferred object	bid incr.	current assignment
0	ϵ	0	0	1	x_1	ϵ	$(1,x_1)$
1	ϵ	2ϵ	0	2	x_2	2ϵ	$(1,x_1),(2,x_2)$
2	3ϵ	2ϵ	0	3	x_1	2ϵ	$(3,x_1),(2,x_2)$
3	3ϵ	4ϵ	0	1	x_2	2ϵ	$(3,x_1),(1,x_2)$
4	5ϵ	4ϵ	0	2	x_1	2ϵ	$(2,x_1),(1,x_2)$

Distributed Optimization

Contract nets and auctions

Weighted matching in a bipartite graph

Assignment problem -- Based on competitive equilibrium

Solution: Add ϵ to the biding increment

$$b_i = u(i,j) - \max_{k|(i,k)\in M; k\neq j} u(i,k) + \epsilon$$

Terminates!

The total number of iterations in which an object receives a bid must be no more than

$$\frac{\max_{(i,j)} v(i,j) - \min_{(i,j)} v(i,j)}{\epsilon}.$$

Corollary *Consider a feasible assignment problem with an integer valuation function* $v : M \mapsto \mathbb{Z}$. *If* $\epsilon < \frac{1}{n}$ *then any feasible assignment found by the terminating auction algorithm will be optimal.*

Distributed Optimization

Social Law

- A restriction on the given strategies of the agents.
- A typical traffic rule prohibits people from driving on the left side of the road or through red lights.
- For a given agent, a social law presents a tradeoff; he suffers from loss of freedom, but can benefit from the fact that others lose some freedom. A good social law is designed to benefit all agents

Distributed Optimization

Social Law

How to compute social laws?

- **Through learning (not covered here)**
- **In an autocratic way (e.g. by design)**